

## Instruções

- Leia este documento **atentamente**. Nele, estão descritos os requisitos mínimos e desejáveis (extras) quanto à implementação do Trabalho Final. Também são apresentadas diretrizes para entrega e critérios de avaliação.

## Objetivo

O objetivo principal do Trabalho Final é exercitar conceitos de programação estudados ao longo do semestre através da implementação de um **jogo de computador** utilizando a linguagem C e a biblioteca gráfica Raylib. Objetivos secundários incluem exercitar as habilidades de pesquisa – uma vez que o aluno poderá utilizar bibliotecas e conceitos que não foram trabalhados em aula para implementar certas funcionalidades – e de trabalho em equipe – uma vez que este pode ser realizado **por duplas** (preferencialmente).

## Temática e Funcionamento Básico

O objetivo é implementar uma versão simplificada do jogo conhecido como Donkey Kong, chamada Donkey Kong - INF (DKINF), utilizando a biblioteca gráfica Raylib. A versão original está disponível no seguinte link <https://freekong.org/>.

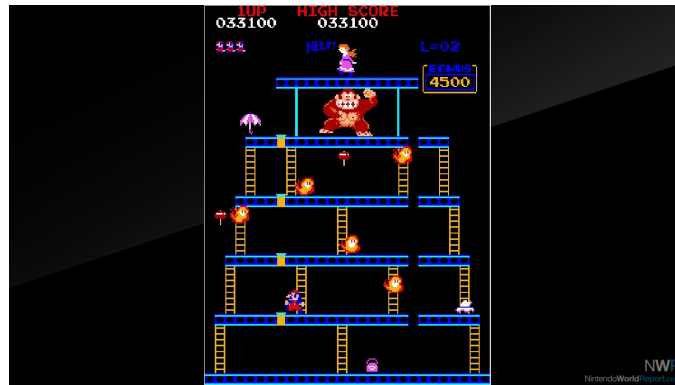


Figura 1: Imagem de exemplo de um nível do jogo original para referência.

Nesta versão, o jogador deve atravessar o cenário, evitando os inimigos e alcançar a porta de saída do mapa — condição que define o término da fase — utilizando as escadas disponíveis, no menor tempo possível (ou, alternativamente, obtendo a maior pontuação, conforme a implementação adotada). O jogo é baseado puramente em mecânicas, não sendo necessária a implementação de história ou narrativa.

O jogo é separado em fases, sendo que cada fase possui um mapa distinto a ser completado. As informações dos mapas estão contidas em arquivos textos "mapaX.txt", sendo que "X" indica o nível atual. No arquivo, há uma matriz de caracteres (com tamanho fixo de 30 linhas por 30 colunas) que indica a posição inicial dos elementos do jogo.

Os caracteres utilizados no mapa são:

| Caractere | Significado                                 |
|-----------|---------------------------------------------|
| P         | Posição inicial do jogador                  |
| E         | Inimigos (1 ou mais)                        |
| Z         | Plataformas                                 |
| S         | Subida de escada                            |
| D         | Descida de escada                           |
| F         | Porta (Final da Fase)                       |
| H         | Escada (Nenhum efeito apenas sprite visual) |

O jogador pode se movimentar horizontalmente utilizando as teclas direcionais esquerda e direita, e subir ou descer escadas utilizando as teclas direcionais verticais.

O movimento horizontal só é permitido quando houver uma plataforma (Z) imediatamente abaixo da posição atual do jogador (linha + 1).

A implementação de pulo é considerada um recurso extra.

Para a implementação básica das escadas, é permitido o deslocamento instantâneo entre suas extremidades: ao posicionar-se sobre um S, o jogador pode pressionar ↑ ou 'W' para ser movido diretamente até a primeira ocorrência de D acima (e vice-versa com ↓ ou 'S'). Soluções mais refinadas serão melhor avaliadas.

Os inimigos se movimentam horizontalmente sobre as plataformas, indo de um lado para o outro. O jogador não possui habilidades de ataque, devendo evitar os inimigos durante sua progressão.

Caso o jogador entre em contato com um inimigo (assuma a mesma posição que o inimigo), ele perde a vida. Por padrão, o jogador possui apenas uma vida; ao morrer, o jogo é reiniciado e o tempo é zerado.

A condição de vitória de cada fase é alcançar a porta de saída (F). Ao entrar nela, o jogador avança para a próxima fase. Ou seja, carrega o próximo mapa.

Ao final do jogo, deve ser exibida uma tela com uma mensagem apropriada (vitória ou *game over*). Em seguida, deve ser apresentada uma tela para inserção do nome do jogador, que será associado à pontuação obtida (independentemente do critério utilizado).

## Visão Geral do Jogo

- O jogador possui as seguintes informações:
  - Localização: coordenadas da linha e coluna (*linha, coluna*)
  - Tempo de execução ou pontuação
  - Estado (ativo ou morto)
- Os inimigos possuem as seguintes informações:
  - Localização: coordenadas de linha e coluna (*linha, coluna*)
  - Direção de movimento (esquerda ou direita)
- A tela do jogo, implementada através da biblioteca Raylib, deve conter:
  - Área de jogo com o mapa carregado
  - Informações de tempo ou pontuação
  - Indicação da fase atual
- **ATENÇÃO:** Tendo em vista que o jogo possui constantes para velocidade do jogador, tamanho de *tile*, altura do pulo (se implementado), etc., cabe ao desenvolvedor tomar decisões que considerar pertinentes quanto aos valores delas. Entretanto, o uso dos chamados “valores mágicos” é fortemente desencorajado. É válido ter em mente que poderá ser pedida na apresentação a alteração de alguma constante acima citada para fins de teste.

## Requisitos Mínimos

Implementações do Trabalho Final devem respeitar os seguintes requisitos mínimos:

- O jogo começa com a apresentação de um menu inicial com as opções:
  1. Novo jogo: inicia uma nova partida
  2. Ranking: apresenta os melhores resultados
  3. Sair: encerra o jogo
- O jogo deve ser implementado em modo gráfico utilizando a biblioteca Raylib. Podem ser utilizados *sprites* (imagens) ou apenas formas geométricas (por exemplo, um quadrado azul representando o jogador e triângulos brancos representando os inimigos). Sprites exemplo podem ser encontradas no Moodle. Grupos podem utilizar outras sprites ou criar as suas.
- O jogo não deve possuir atrasos artificiais nas ações do jogador.
- O mapa do jogo deverá ser carregado a partir de arquivos texto nomeados "mapaX.txt", onde X indica a fase correspondente.
- O placar deve ser salvo e carregado a partir de um arquivo placar.bin. Esse contém até 10 nomes de jogadores e suas respectivas pontuações anteriormente salvas. O arquivo placar.bin será formado por um arranjo de dez structs, dados por:

```
typedef struct tipo_placar
{
    char nome[20];
    int time;
} TIPO_PLACAR;
```

Quando o jogo for encerrado, o programa deve verificar se a pontuação do jogador atual é uma das dez melhores. Caso afirmativo, perguntar o nome do jogador e atualizar o arquivo placar.bin. O arquivo deve ser mantido sempre ordenado em ordem decrescente de pontuação. O sistema de ranking deve ser baseado em tempo (menor é melhor) ou pontuação (maior é melhor). A escolha deve ser consistente com o campo armazenado no placar.bin.

- O programa deve ser capaz de interpretar corretamente a matriz do mapa com tamanho fixo de 30 linhas por 30 colunas.
- O jogo deve permitir múltiplas fases, carregadas dinamicamente. O jogo deve ser entregue com **pelo menos 2 fases** diferentes dos exemplos. E um mapa será disponibilizado no dia da apresentação para testes.
- A interação do jogador com o jogo se dá ao pressionar teclas específicas. As ações e teclas são as seguintes:

| Tecla                | Ação                 |
|----------------------|----------------------|
| ← / → ou ('A' / 'D') | Movimento horizontal |
| ↑ ou ('W')           | Subir escada         |
| ↓ ou ('S')           | Descer escada        |
| TAB                  | Pausar o jogo        |

- O jogo deve possuir sistema de pausa funcional, interrompendo completamente a execução da lógica do jogo e permitindo posterior volta a partir do mesmo ponto e mesmas condições. O menu de pausa deve possuir ao menos as seguintes opções:

| Opção          | Significado                               |
|----------------|-------------------------------------------|
| Continuar      | Retoma o jogo do ponto em que foi pausado |
| Voltar ao menu | Retorna ao menu principal                 |
| Sair           | Fecha o jogo                              |

- O jogador não deve atravessar plataformas e nem caminhar fora de plataformas.
- Os inimigos devem se movimentar sobre as plataformas sem atravessar obstáculos.
- Ao colidir com inimigos, isto é, quando o jogador se encontrar na mesma posição que o inimigo, o jogo deve reiniciar (ou consumir vida, caso implementado como extra).
- O jogo deve possuir sistema de ranking baseado em tempo ou pontuação.
- O jogo deve funcionar corretamente mesmo com modificações externas nos arquivos de mapa (leitura dinâmica).

Tabela 1: Exemplo de Arquivo de Texto que Define o Nível (em formato de tabela)

[illegible]

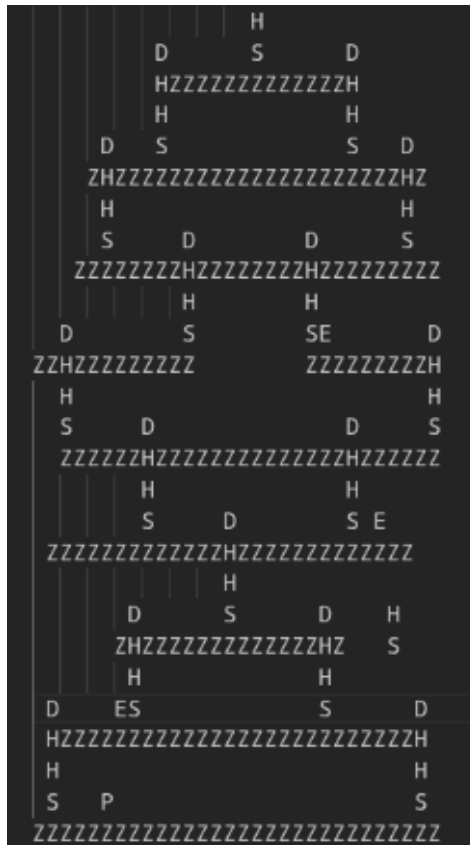


Figura 2: Exemplo de arquivo de mapa sendo exibido no jogo.

## Requisitos Extras

Para os alunos motivados, as seguintes tarefas extras são propostas. Embora opcionais, essas tarefas podem melhorar a avaliação final do seu trabalho, caso não tenham conseguido implementar corretamente algum dos requisitos básicos.

- Implementação de pulo: o jogador pode pressionar uma tecla para saltar e sobrepor obstáculos ou evitar inimigos.
- Sistema de múltiplas vidas: ao invés de reiniciar imediatamente ao colidir com um inimigo, o jogador perde uma vida e recomeça a fase, sendo o jogo encerrado apenas quando todas as vidas se esgotarem.
- *Power-ups* no mapa: itens espalhados pelo cenário que conferem vantagens temporárias ao jogador (invencibilidade, velocidade aumentada, pontos extras, etc.).
- Diferentes tipos de inimigos: implemente mais de um tipo de inimigo, desde que os diferentes tipos ajam de maneira distinta. Por exemplo, inimigos que perseguem o jogador, ou que se movem em padrões diferentes.
- Animações para jogador e inimigos: utilize *spritesheets* para animar os personagens durante o movimento.
- Sons e música: adicione efeitos sonoros para eventos do jogo (colisão, subida de escada, vitória, etc.) e música de fundo.
- Interface gráfica mais elaborada: texturas, *backgrounds*, elementos visuais adicionais, etc.
- Sistema de física mais complexo: gravidade, inércia e outras mecânicas que tornem a movimentação mais realista.
- Sejam criativos, conversem com o professor ou professora e implementem suas ideias!

## Entrega

O trabalho será entregue em **3 (três) etapas**:

1. Indicação dos grupos: Definição dos integrantes via Moodle. **O trabalho deverá ser realizado preferencialmente em duplas**. O grupo deve informar os nomes de seus integrantes até o prazo definido, através da plataforma Moodle.
2. Entrega do código: O grupo deverá submeter via Moodle um arquivo zip cujo nome deve conter o(s) nome(s) do(s) aluno(s). O arquivo zip deve conter:
  - Um *breve* relatório contendo a descrição do trabalho realizado, incluindo a especificação de como os elementos do jogo foram representados, como foi implementada a interação dos componentes, bem como as estruturas e funções utilizadas e uma explicação de como usar o programa. O relatório pode ser simples e objetivo, mas deve conter todas essas informações.
  - Os códigos-fonte devidamente organizados e documentados (.c).
  - Os códigos executáveis já compilados e prontos para serem executados. O executável deve ser compatível com o ambiente utilizado na disciplina (ou acompanhado de instruções de compilação).
3. Apresentação: Demonstração do funcionamento do jogo e explicação do código.
  - O trabalho será obrigatoriamente apresentado durante a aula de apresentação. Todos os membros do grupo devem saber responder perguntas sobre qualquer trecho do código e serão avaliados separadamente.

## Avaliação

Os seguintes itens serão considerados na avaliação do trabalho: estruturação do código em módulos, documentação geral do código (comentários, indentação), “jogabilidade” do jogo e atendimento aos requisitos definidos. A divisão da pontuação é a seguinte:

1. Habilidade em estruturar programas pela decomposição da tarefa em subtarefas, utilizando subprogramação (funções e módulos) para implementá-las **(2 pontos)**.
2. Organização e documentação de programas (indentação, utilização de nomes adequados para variáveis e constantes, abstração dos procedimentos para obter maior clareza, uso de comentários no código) **(1 ponto)**.
3. Domínio na utilização de tipos de dados simples e estruturados (arranjos e estruturas) e passagem de parâmetros por cópia e referência **(2 pontos)**.
4. Atendimento dos requisitos do enunciado do programa: menus, elementos gráficos esperados, interação, movimentação de personagens, funções dos menus, etc. **(5 pontos)**.

## Dicas

- Use uma IDE (como o VS Code ou Code::Blocks) para desenvolvimento.
- Utilize (e abuse de) structs para representar os elementos dinâmicos do jogo. Lembre-se que você pode colocar uma struct dentro da outra, como por exemplo, uma struct *Posição* com x e y (ou linha e coluna) dentro de uma struct *Jogador*.
- Separe a lógica do jogo com a renderização na tela. Uma dica é sempre primeiro atualizar o estado das variáveis a partir das entradas do usuário e só após isso desenhar o que for necessário.
- Se antecipe e vá desenvolvendo o jogo conforme o tempo passa até a data de entrega. É útil ir implementando item por item e checando se o que está sendo desenvolvido está correto. Por exemplo, primeiramente faça a leitura e renderização do mapa, depois a implementação do jogador e seu movimento, depois os inimigos e assim por diante. Modularize seu jogo para evitar propagação de erros!
- Para controlar a velocidade do jogo, utilize a função `SetTargetFPS` da Raylib.

- Considere o uso de repositórios remotos de código (*Github*, *Gitlab*) com controle de versão para desenvolvimento em equipe.
- Recorra ao monitor para sanar dúvidas gerais sobre instalação de bibliotecas, dúvidas a respeito do uso da Raylib, uso de repositórios (*Github*), uso de ferramentas e implementação de funcionalidades associadas ao jogo.
- É comum, à medida que formos estudando novos conteúdos, surgirem ideias de como utilizar esses conteúdos para melhorar o jogo. Isso acaba gerando modificações constantes no jogo. Isso é esperado e bem-vindo.
- Sugestão de desenvolvimento incremental:
  1. Movimentação do jogador
  2. Leitura e renderização do mapa
  3. Colisão com plataformas
  4. Inimigos
  5. Sistema de fases
  6. Menu e ranking

## Observações

Este trabalho deve refletir a solução individual do grupo para o problema proposto. Casos de plágio (mais de 75% de similaridade – incluindo entregas de outros semestres) serão tratados com severidade e resultarão em anulação da nota do Trabalho Final (vide programa da disciplina). Para detectar e quantificar o plágio no código, será utilizado o *software* MOSS (<http://theory.stanford.edu/~aiken/moss/>).

## Uso de Inteligência Artificial

O uso de ferramentas de Inteligência Artificial (IA) é permitido como apoio ao desenvolvimento, desde que utilizado de forma responsável e complementar.

Ressalta-se que o objetivo principal da atividade é o aprendizado e a compreensão dos conceitos de programação. Portanto, o grupo é inteiramente responsável pelo código submetido e deverá ser capaz de explicar, de forma clara e completa, o funcionamento de todas as suas partes.

O uso indiscriminado de código gerado por IA, sem entendimento, poderá impactar negativamente a avaliação do trabalho.